本形學
Ontypology

- 從異同到分合，從分合到形，從形到理解這條路，大腦走了億萬年。
- 邏輯學百年前是哲學，當看清楚後是顯學(科學)
- 今探討「形」(大腦概念) 是哲學，本形學將「形」，看清楚轉成顯學(科學)，並實践化 Mind AI 。

本形学 Ontypology ★

异同→分合→形→理解→显学(科学)



ONTYPOLOGY 本形學 × MIND AI 平台

IC 設計大腦

準確使用方法完全方案

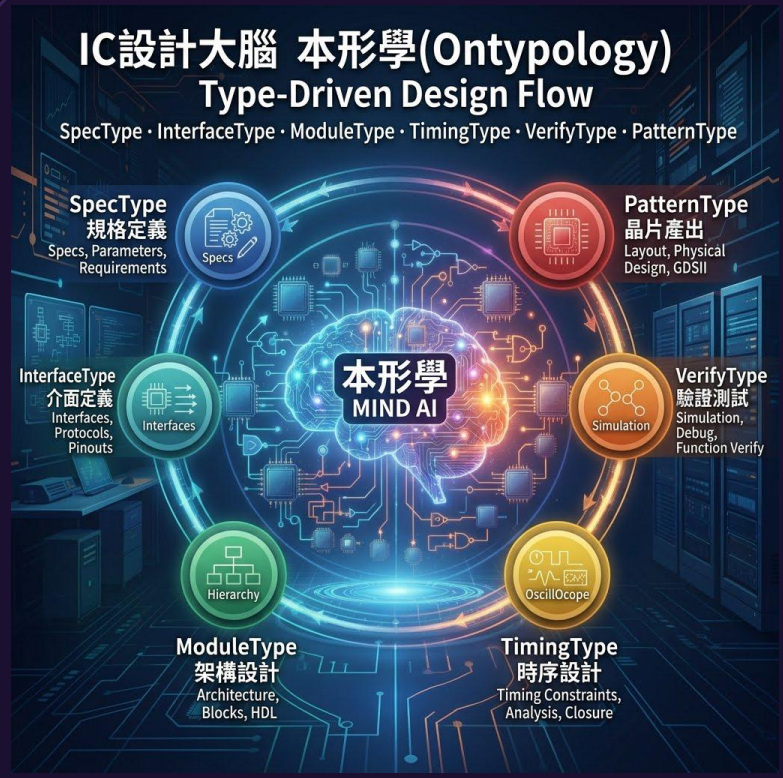
它不解釋理論本身，而是給出一套可照做的準確使用方法：在 Mind AI 平台上，把空間本形與時間本形完整運用於 IC 設計流程，從規格文件一路走到通過驗證的 Verilog RTL。

S1	S2	S3	S4	S5	S6	S7
規格	介面	模組	骨架	RTL	驗證	演化

七步驟主線 · 四層準確性保證 · 六種形 · 八支 Verilog 範例

判準可判定 · 過程可回溯 · 產出可驗證 · 經驗可沉澱

使用手冊 + 完整方案 ｜ 附錄 A–D：提示詞範本 · 檔案清單 · 術語對照 · 檢核表



本形學 × Mind AI 知識圖

SpecType 規格形

InterfaceType 介面形

ModuleType 模組形

TimingType 時序形

VerifyType 驗證形

PatternType 模式形

一本同時是「使用手冊」與「完整方案」的書

使用手冊面

給誰 | 第一線設計 / 驗證工程師

讀法 | 隨查隨用：直接抄範本、照檢核表打勾，拿到規格就能沿 S1–S7 走完一趟

章節 | 第五章 S1–S7、第六章 Verilog、附錄 A/D

產出 | 提示詞輸出、RTL、驗證報告

完整方案面

給誰 | 架構師、計畫主持人、平台管理者

通讀：理解雙本形驅動的完整結構與治理機制

章節 | 第二～四章、第七～八章

產出 | 形庫、檢核體系、成熟度路線

本書的四條使用契約

契約一 理論不重述

本形學與 Mind AI 的理論本體不在本書範圍；所有術語皆作為「已知的操作語彙」直接使用。

契約二 產出以 RTL 為準

交付物為 Verilog/SystemVerilog 程式碼與型化證據；平台程式、工具鏈開發不在範圍。

契約三 譯名規約：形 = Type

英文一律用 Type，不用 Form：SpecType / InterfaceType / ModuleType / TimingType / VerifyType / PatternType。

契約四 全程以形說話

任何產出都要能命名為某個形、附上 ∩(T) 判準、並留下形證據；說不出形的產出視為未完成。

準確不是願望，而是四個可稽核的條件

01
判準可判定

每個形都有 ∩(T) 交集判準，新設計可被「判入 / 判出」，不靠感覺。

02
過程可回溯

從規格到 RTL 的每個決策，都能沿演繹骨架回溯到源頭判準。

03
產出可驗證

每份 RTL 皆有對應驗證形與形覆蓋率 TypeCov，證據可重現。

04
經驗可沉澱

每次專案的模式與誤用都回填形庫，下一次從形庫出發而非從零開始。

四條件同時成立，才稱得上「準確使用」

缺任何一項，產出即視為未完成：說不出形的產出，不是這套方法的交付物。

對應本書的六種形：SpecType · InterfaceType · ModuleType · TimingType · VerifyType · PatternType，每一種都要有判準與證據。

四種角色，四條路：先選起點，再讀必讀

系統架構師

起點

第二章

必讀

第三章、第四章、5.1–5.4

主要產出

SpecType / ModuleType / 演繹骨架

RTL 設計工程師

起點

第五章

必讀

4.3、5.5、第六章

主要產出

Verilog RTL 與時序契約

驗證工程師

起點

5.6

必讀

第七章、6.8

主要產出

testbench、TypeCov 報告、反例清單

計畫 / 平台管理者

起點

第一章

必讀

第七章、第八章、附錄

主要產出

形庫治理、檢核體系、成熟度計畫

範圍邊界：本書明確不做的三件事

不重述理論本體

異同→分合→形、屬性 / 屬性值、BOM/SOM、框架四步驟、三括號與 ∩(T)，以及搜尋、推理、設計模式、自我演化，屬理論文獻。本書只給「在 IC 設計裡怎麼用」。

不寫系統實作

Mind AI 平台的程式實作、工具鏈開發、介面整合程式碼不在範圍；本書的程式產出一律為 Verilog / SystemVerilog。

不依附其他方法學

本書自成體系：以本形學為唯一理論基礎、以 Mind AI 為唯一平台底座，所有方法皆由此推導。

全書共同語言：六種形，英文一律 Type

SpecType
規格形

把需求收容成可判定的形

產出｜SpecType 清單、屬性—屬性值表

InterfaceType
介面形

定義模組對外的訊號契約

產出｜訊號表（表頭即形）

ModuleType
模組形

定義內部組成與依存

產出｜BOM / SOM 結構表

TimingType
時序形

定義時間上的行為契約

產出｜狀態形、波形契約

VerifyType
驗證形

定義如何取證與判定

產出｜testbench、TypeCov 覆蓋表

PatternType
模式形

沉澱可重複套用的編碼模式

產出｜形庫中的模式卡

譯名稽核　任何文件、提示詞、程式註解出現「Form 代指形」的用法，皆視為錯誤並須當場更正；口語可說類別 / 型別，書面產出一律回歸 Type 術語體系。

架構總覽：兩本形一支點

全書的總骨架

左翼空間本形負責「結構正確」，右翼時間本形負責「過程正確」，中央 Mind AI 承載雙翼：LLM 肩負歸納統計的語言能力，Mind Model 肩負演繹推理的智慧能力，形庫保存兩翼共用的形。

回答的問題

空間｜這個設計的結構對不對？

時間｜產生這個設計的過程對不對？

錯誤的型態

空間｜部件遺漏、介面破洞、判準不可判定

時間｜跳步、無來源推理、經驗流失

對應的形

空間｜SpecType / InterfaceType / ModuleType

時間｜TimingType / VerifyType / PatternType

準確性層

空間｜第一層 ∩(T) 判準、第二層演繹骨架

時間｜第三層形式驗證、第四層形證據鏈



圖 2-1　雙本形驅動架構總覽：空間本形定義結構正確、時間本形定義過程正確、Mind AI 承載全流程

一支點：Mind AI 平台

平台把兩翼語彙接成一條產線：spec → verified RTL。空間本形給出形與判準，時間本形給出步驟與推理，形庫是兩者共用的記憶。

空間本形的七條運用準則

01 異同→分合→形

逐句找差異；同性質合、不同性質分，每組即一個 SpecType

02 屬性 / 屬性值

欄位 = 屬性、填值 = 屬性值；屬性須貫穿全部個體

03 BOM 部件清單

只列「拆下來還認得自己」的東西；拆不下的進屬性

04 SOM 空間清單

BOM 疊上依存與位置；必要件與選配件必須明示

05 框架四步驟

辨差異→對照失敗→取名分維度→新個體判維度

06 三括號形集合

( ) 參數容器、{ } SOM 組成、[ ] 抽象家族

07 ∩(T) 交集判準

x ∈ T ⇔ F(T) 全成立；成員由判準動態生成

準則不是理論重述

而是「當你做 X 時，照這樣做」的操作律。任何一列有空白欄位，就是萃取未完成；形必有名、有 ID。



圖 2-2　空間本形 → IC 設計的運用映射

第三、四章把七條準則拆細

形萃取、屬性表、BOM/SOM、雙軸定位、三括號、∩(T) 撰寫、形庫結構，逐項給出操作程序與判斷對照。

時間本形的七條運用準則

01 形時間函式搜尋

先搜尋形庫，以 ∩(T) 為查詢；未命中才新建

02 設計搜尋與萃取

萃取 PatternType 而非程式碼片段：片段無判準

03 設計模式

RTL 以模式拼裝：FSM、移位、除頻、FIFO

04 邏輯生成與應用

順序固定：判準→骨架→RTL，禁止自由生成

05 推理與思考法

時序因果、除錯逆向、效能模擬、取樣統計

06 重構與重組

介面形與 ∩(T) 凍結，只動內部 BOM/SOM

07 持續學習與自我演化

結案三件事：模式入庫、誤用入表、判準修訂



圖 2-3　時間本形 → IC 設計的運用映射

時間本形管的是「過程」

搜尋是設計的第一拍；模式先於程式碼；骨架先於 RTL；外在形不變才叫重構；三件事做完才准結案。

七條準則對應 S1–S7

第四章逐節展開思維動作，第五章做成可照做的操作手冊：輸入、動作、輸出、提示詞範本、檢核表。

四層準確性保證：由內而外逐層封裝

① ∩(T) 判準　這個設計算不算這個形？

機制｜特徵集合逐項判定，可判入也可判出

失敗｜判出即拒收，退回重做形定義

② 演繹骨架　這個結論從哪裡來？

機制｜自判準演繹出結構，每步附推導鏈

失敗｜無來源的結論一律刪除

③ 形式驗證　波形真的符合契約嗎？

機制｜assertion、模擬比對、契約波形核對

失敗｜反例入清單，修到反例清空

④ 形證據鏈　這件事做過了嗎？可信嗎？

機制｜每步操作留型化證據，可回溯審計

失敗｜證據斷裂 = 未完成，不得入庫



圖 2-4　四層準確性保證：由內而外逐層封裝

四層不是事後檢查清單，而是嵌在流程裡的順序

寫需求時先寫判準（一），推導結構時留下來源（二），生成 RTL 後做驗證（三），全程留證（四）。越早的層越便宜——第一層的一句判準，勝過第三層的三天除錯。

形驅動設計七步驟：S1 到 S7

S1　規格形萃取

SpecType → 清單、屬性—屬性值表

S2　介面形定義

InterfaceType → 訊號表、介面 ∩(T)

S3　模組形分解

ModuleType → BOM/SOM 結構表

S4　演繹骨架

骨架文件 → 演繹骨架、契約表

S5　時序形與 RTL 生成

TimingType · PatternType → Verilog、波形契約

S6　驗證形建構

VerifyType → testbench、TypeCov 報告

S7　形證據鏈與自我演化

形庫增量 → 形證據鏈、形庫版本

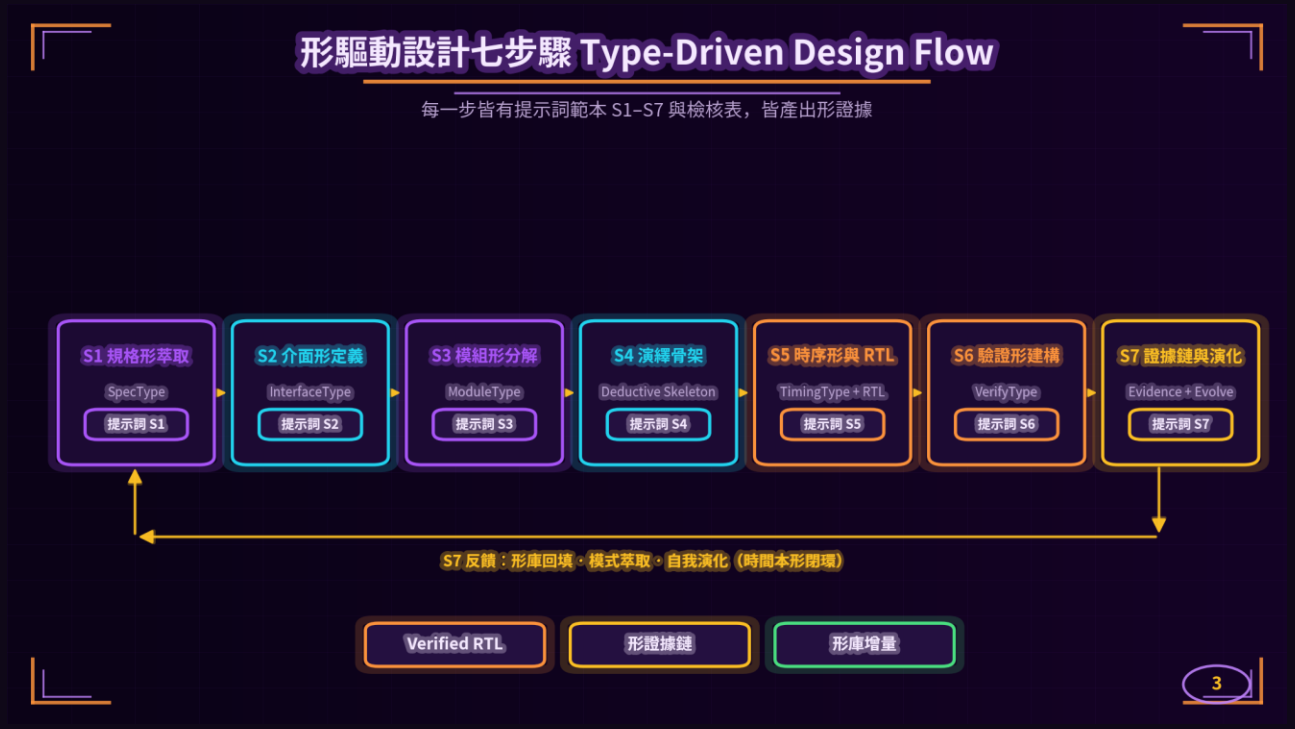


圖 2-5　Type-Driven Design Flow 七步驟與 S7 反饋閉環

閉環在哪裡

S7 的輸出回填形庫，成為下一輪 S1 的「既有形庫索引」與 S5 的模式來源。設計一次專案，形庫長大一輪。

五要素 × 以形交接

每一步都有輸入、動作、輸出、提示詞範本與檢核表；步驟之間以「形」交接，上一步的輸出形就是下一步的輸入形，因此任何一步都能獨立驗收、獨立重跑。

形萃取四拍 × 表頭即形

形萃取：在規格文本中畫出邊界

運用要點：規格書不是形，規格書裡的「差異」才是形的原料。

第一拍　辨異同

逐句比對規格文字，標記「不一樣的地方」：協定、模式、時脈域都是異同。

第二拍　做分合

同性質的差異合為一組，不同性質的切開；寬度跟協定不是同一回事。

第三拍　命名成形

每組取形名、編形 ID，寫下 F(T) 特徵集與 ∩(T)：形必有名、有 ID。

第四拍　回填全體

形一旦成立，回頭為所有已出現的個體補值——這是最多人偷懶的一步，也是結構完整的關鍵。

判斷對照：差異能對上既有維度就只加屬性值，不無限開新形。

表頭即形：訊號表就是介面的形

運用要點：訊號表不是文件格式；表頭是形，資料列是實體。

signal_name	clk / rst_n / data_in	屬性：名稱
direction	input / output / inout	屬性：方向
width	1 / 8 / [DATA_WIDTH-1:0]	屬性：寬度
domain	sys_clk / spi_clk / async	屬性：時脈域
reset_value	0 / 1 / X	屬性：復位值

兩種變動不對稱

新增一列（新案例）輕巧：形不動，隨手可做。

新增一欄（新維度）沉重：形變大，所有既有列都要補值。

評審標準提問　看到「順手加一欄」的提案要停下來問：這是新個體，還是新維度？欄位能貫穿全部個體，才比較得了。

BOM / SOM 分解：一條判準分三類

「拆下來之後，它還認得自己嗎？」

這條判準把模組的元素精確切成三類。BOM 只列清單，SOM 再疊上依存與位置。

① 拆得下、認得自己 → BOM 部件

uart_tx 收發器：可獨立實例化、可獨立驗證，是必要件。

② 拆不下來 → 屬性

時脈頻率 50MHz：只能描述「這個東西是什麼樣子」。

③ 可替換 → ( ) 參數容器

CLKS_PER_BIT：換值不換身份；sync_fifo 緩衝是選配件，拿掉只損功能。

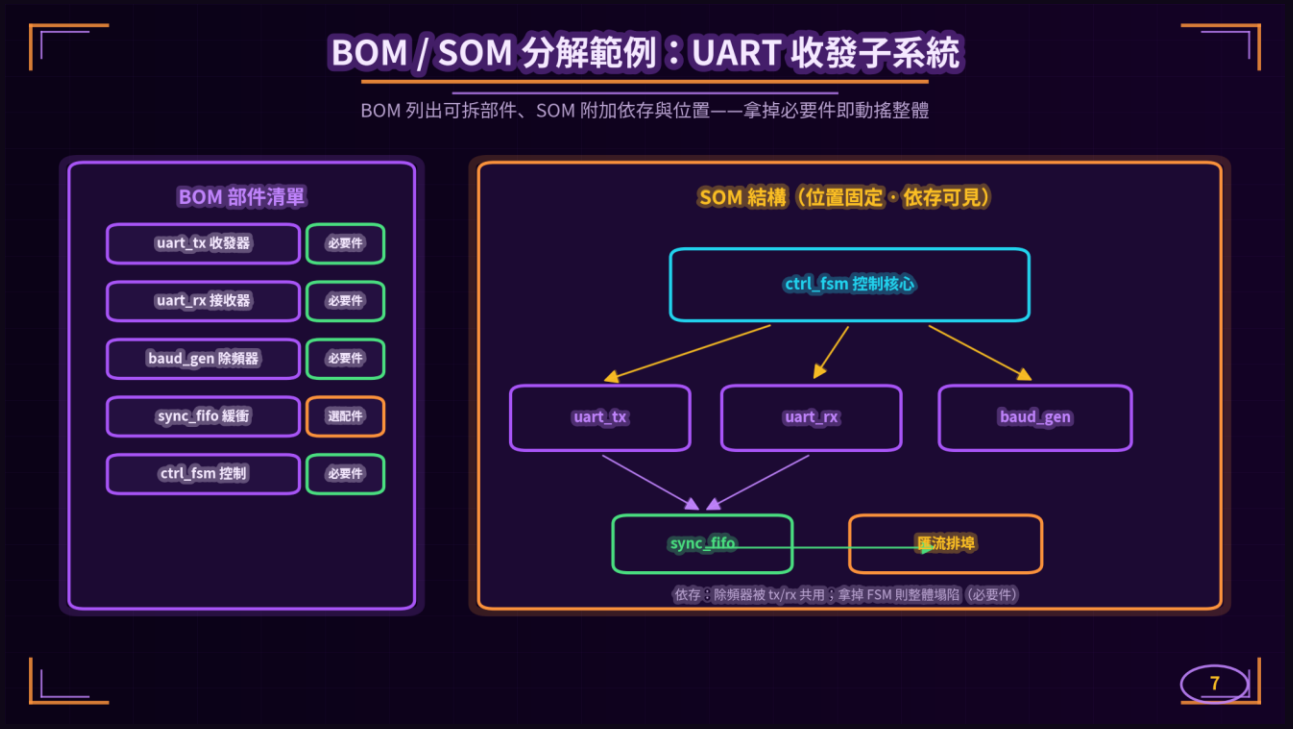


圖 3-1　BOM/SOM 分解範例：UART 收發子系統

SOM 的價值在依存

標註哪些部件互相依存、哪些是必要件（拿掉整體塔陷）、哪些是選配件（拿掉只損功能不失身份）。必要件與選配件必須明示。

X 軸抽象 × Y 軸分解・三括號形集合



圖 3-2　X 軸抽象與 Y 軸分解在規格轉換中的運用



圖 3-3　三括號形集合在 IC 設計中的運用

雙軸定位：兩個不同的問題

X 軸問「它跟什麼是同一類」，往上收斂成 [ ] 抽象家族（UART、SPI、I²C 同屬序列介面形）。

Y 軸問「它由什麼組成」，往下拆成 { } 組成集合。

S2 以 X 軸為主、S3 以 Y 軸為主；只做單軸的設計，檢核表直接不通過。

三括號是三種集合語意，混用會把層次拍平

( ) 身份獨立性 | 參數容器；錯：把參數值不同當成不同模組。

{ } 元素間組成結構 | BOM/SOM；錯：當無序清單隨意增刪。

[ ] 特徵交集生成規則 | IP 家族、模式庫；錯：用清單列舉成員、寫死數量。

∩(T) 交集判準撰寫法：壞判準 vs 好判準

壞判準 | 只能判入	好判準 | 也能判出
可判定性 「一個高效的 FIFO」	**逐條可判定** 「深度≥4、同時讀寫計數不偏移、full 時拒寫」
完備性 只列功能、漏掉時序	**四類特徵齊備** 特徵覆蓋功能 + 介面 + 時序 + 復位行為
開放性 「必須是以下三者之一」	**成員由判準生成** 「通過全部特徵者即為成員」，歡迎新成員
反例 從不思考判出的情況	**明寫反例** 同拍滿載寫入被拒——這是判出反例

合格判準的三件東西：特徵集合 F(T) · 判準式 x ∈ T ⇔ F(T) 全成立 · 開放性聲明

形庫 Type Library 的資料結構

表頭即形：四表結構

每張表的表頭就是一個形（欄位 = 屬性），每一列就是一個被指認的實體。主鍵是實體身分的憑證——屬性可以改，主鍵不能換，否則全部從表斷鏈。

SpecType 表 PK: SpecType_id

名稱、F(T) 特徵集、∩(T) 判準、來源規格

ModuleType 表 PK: ModuleType_id · FK: SpecType_id

BOM、SOM 依存、參數容器、必要/選配

VerifyType 表 FK: ModuleType_id

覆蓋目標、反例清單、證據連結

PatternType 表 PK: PatternType_id

模式卡、適用判準、已知誤用

形庫腐化的頭號原因

把新增欄當新增列做。新增列 = 形不動，隨時可做；新增欄 = 形變大、必須回填整個形庫。

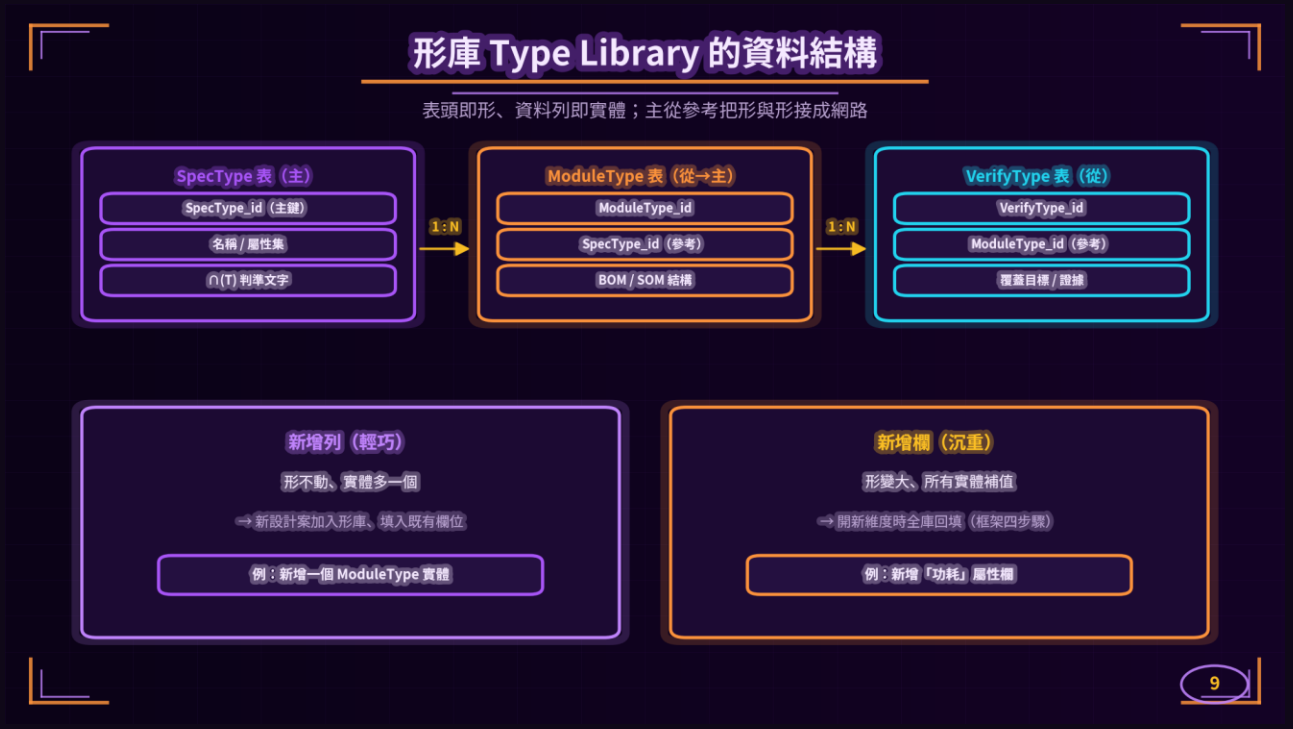


圖 3-4　形庫的資料結構：表頭即形、主從參考連網

「關聯」在形庫裡的樣子

表與表之間以主從參考相連：ModuleType 表的 SpecType_id 欄位只做一件事——指向 SpecType 表的某個形。入庫門檻：∩(T) 齊備 + 證據鏈無斷點 + 形主簽核。

流程節拍對位：每一拍都要留下一個痕跡

空間本形保證「長對結構」，時間本形保證「走對過程」。流程上的每一拍都要對上一個模式、一個思維動作、一個可查的產出痕跡——**沒有痕跡就無法回溯，無法回溯的過程無從談準確。**

01
立項 / 進案
形時間函式搜尋｜在形庫檢索可複用的形
痕跡：搜尋紀錄（判準、命中 / 未命中）

02
規格分析
解題·分析｜因果推理拆解需求依存
痕跡：SpecType 萃取記錄

03
架構設計
設計·研發探索｜第一性原理 + 水平思考
痕跡：方案比較表、選型紀錄

04
詳細設計
設計模式·邏輯生成｜模式選定→骨架拼裝
痕跡：模式卡引用、演繹骨架

05
驗證除錯
推理（逆向·模擬·統計）｜反例回溯
痕跡：反例清單、診斷報告

06
結案
重構重組·持續學習｜模式萃取、檢核項沉澱
痕跡：形庫新版本、檢核表更新

4.2 形時間函式搜尋：設計的第一拍

任何設計先搜尋形庫，命中即複用並記錄出處；未命中才允許新建。

以 ∩(T) 為查詢，不以名稱為查詢

帶著判準進形庫：把需求轉成特徵條件，逐條比對候選形的 ∩(T)。找「長得像」而不是「名字像」——名稱會漂移，判準不會。

命中率不滿時，只有兩種處置

部分命中 → 複用並標註差異，記為「形的特化」；完全未命中 → 才允許新建，且紀錄必須附上「為何既有形不可用」的理由。

搜尋與萃取是雙向的：進案時搜尋、結案時萃取，一來一往構成形庫的呼吸。少了這一來一往，形庫就是一座靜態檔案庫；有了它，形庫才會隨著專案長大——**這就是時間本形「持續學習」在流程上的落點。**

PatternType 模式卡：模式先於程式碼

時間本形的「設計模式」在 IC 落地為 RTL 編碼模式庫。生成任何一段 RTL 之前，先選模式；程式碼只是模式的實體化。下表是形庫 PatternType 應當預置的最小模式集——每張卡都自帶適用判準與已知誤用。

FSM 二段式

適用　狀態少、輸出純組合

誤用　輸出含暫存路徑時誤用 → 輸出毛刺

FSM 三段式

適用　輸出需暫存、時序收斂

誤用　狀態少時過度使用 → 面積浪費

除頻器

適用　由高頻生成低頻節拍

誤用　直接以計數器輸出當時脈 → 時脈偏斜

移位暫存器

適用　序列化 / 反序列化

誤用　移位與計數分離失控 → 位元錯位

同步 FIFO

適用　跨模組緩衝、速率匹配

誤用　深度不覆蓋突發 → 溢位

CDC 雙正反器

適用　跨時脈域單訊號

誤用　多訊號分別同步 → 拜占庭失敗

邊緣偵測

適用　脈波化、事件捕捉

誤用　用在非同步訊號 → 漏觸發

仲裁器

適用　多請求單資源

誤用　無固定優先 → 飢餓

為什麼要把「已知誤用」寫進模式卡？　因為模式卡記的不是「怎麼寫」，而是「什麼時候不能用」。判準讓形可辨識，誤用欄讓形可安全複用——兩者同時齊備，這張卡才進得了形庫。

生成的紀律：判準 → 骨架 → RTL

這個順序是時間本形最強的紀律，也是準確性的核心分工：語言能力交給 LLM，智慧能力交給 Mind Model。

STEP 1

∩(T) 判準

空間本形的判準必須先存在

全部特徵條件齊備，才准許往下走；**生成有裁判，才有準確。**

STEP 2

演繹骨架

由 Mind Model 演繹生成

模組層級 · 介面契約 · 時序骨架；**每個結論都有來源編號。**

STEP 3

Verilog RTL

由 LLM 翻譯骨架成語言表面

RTL 結構與骨架一一對應；**時序契約與波形一致。**

LLM　負責歸納統計

把骨架與選定的 PatternType 模式翻譯成程式碼的「語言表面」。**它不負責決定「結構是什麼」。**

Mind Model　負責演繹

從 ∩(T) 出發演繹出骨架，決定模組邊界、介面契約與時序結構。**智慧能力在這一邊，不在語言模型那邊。**

禁止自由生成

任何跳過骨架、由規格文本直接生成的 RTL，不論看起來多完整，都不得進入 S6——它無法回溯、無法驗收，出錯時也無法定位是規格錯、骨架錯還是程式錯。

推理與思考法怎麼落地，重構為什麼永遠安全

時間本形不只給出模式與節拍，還給出一整套推理與思考法。左欄是五種運算方向，右欄是三種探索姿態；右下則是重構的安全邊界。

推理　五種運算方向

因果推理　時序收斂、根因分析
實例：setup 違例 → 定位組合路徑的因

果因推理　由失敗波形倒推設計缺陷
實例：ACK 錯誤 → 倒推相位契約破口

逆向推理　除錯、驗證收斂
實例：由 assertion 失敗回溯到骨架結論

模擬推理　效能 / 功耗預估
實例：以代理指標預估吞吐上界

統計推理　取樣、覆蓋率、良率
實例：三點多數決、隨機激勵覆蓋

思考法　三種探索姿態

第一性原理思考　架構決策
回到「為何需要此介面」重推方案

水平思考　替代方案探索
同功能換架構：串列 vs 並列

垂直思考　層級細化
系統 → 模組 → RTL 逐層下鑽

重構的契約：外在形不變

凍結　介面形、∩(T)、時序契約全部凍結。

可動　只動內部的 BOM / SOM：合併狀態、重排管線、替換模式。

停損　一旦「不動介面就改不下去」，停止重構，啟動形變更流程（回 S2 重走）。

這條邊界讓重構永遠安全：驗證形不作廢，只是重新跑一次確認。

持續學習與自我演化：結案必做的三件事

自我演化不是神秘學，而是寫進檢核表的三件事

01 模式萃取

本專案中重複出現兩次以上的程式結構，萃取成 PatternType 模式卡入庫。

02 誤用沉澱

本專案踩過的坑，寫成「已知誤用」欄位與檢核項——下次別人不必再踩。

03 判準修訂

驗證中發現判準不可判定或不完備的，修訂 ∩(T) 並記錄版本。

三件事做完，才允許結案。

這是時間本形「持續學習」在專案管理上的最小實踐：學習不靠口號，靠三個有產出痕跡的結案動作。

閉環的順序　設計 → 驗證 → 學習 → 形庫演化 →（下一個設計的）搜尋

「學習」這一環有明確的三件交付物，所以它可以在檢核表上打勾；打不了勾的學習，下一專案就不會發生。

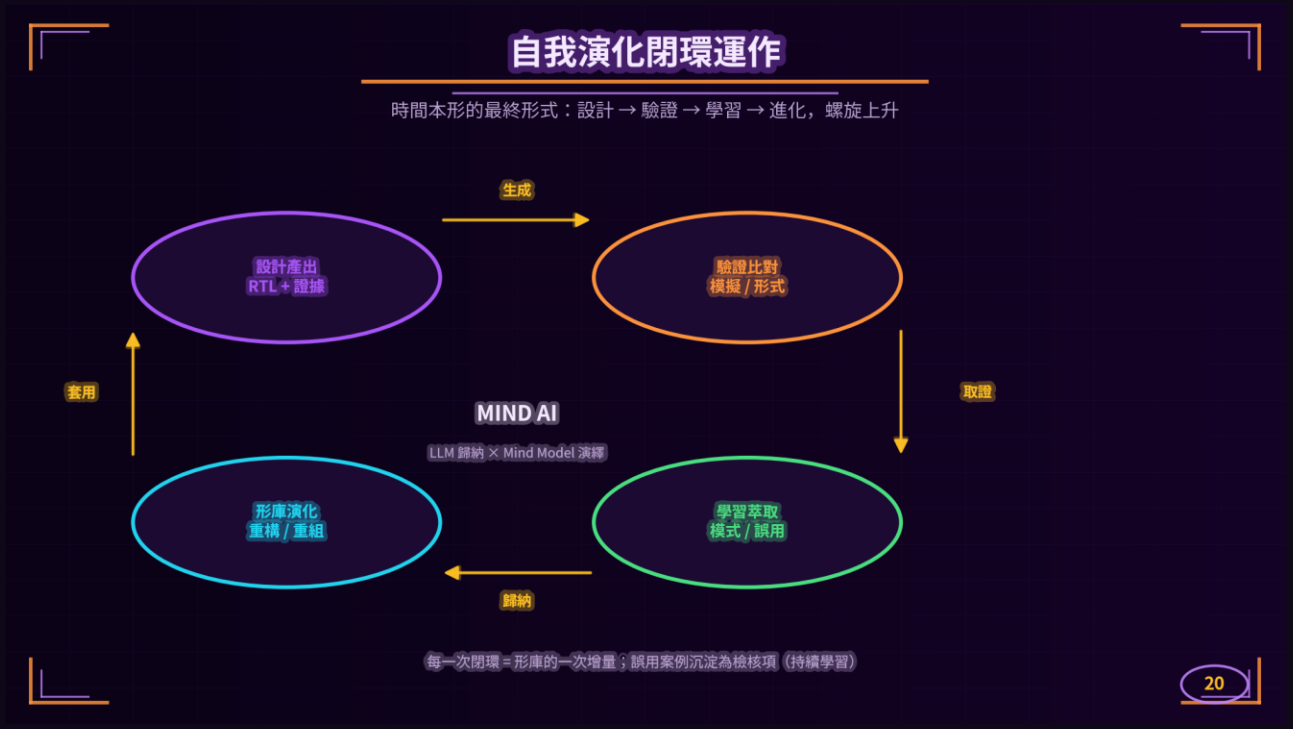


圖 4-1　自我演化閉環：設計 → 驗證 → 學習 → 形庫演化

形庫不是起手式，是收尾的產物

進案時以 ∩(T) 搜尋、結案時以 ∩(T) 入庫，形庫因此每一個版本都比上一個版本更好用。這是時間本形「自我演化」在 IC 團隊裡唯一可稽核的形式。

提示詞範本的使用總則：五條紀律

全書的操作核心：七步驟 × 五區塊

一步一範本

S1–S7 各司其職；禁止把兩步的任務塞進同一則提示詞——混步的輸出無法驗收。

判準先行

任何範本都先要求輸出 ∩(T) 判準再談其他——生成有裁判，才有準確。

格式即契約

範本規定輸出格式（形表格、證據欄位）；格式缺欄 = 輸出不合格，退回重跑。

檢核表收尾

每次範本輸出後接續跑檢核表；打勾不是形式，是 S 驗收的證據。

證據即產物

無證據欄位的輸出視為未完成；證據欄位留空 = 這一步還沒做。

每個步驟都固定五個區塊　定位與本形運用 | 輸入 / 輸出 / 判準 | 操作程序 | 提示詞範本 | 檢核表與常見誤用。



圖 5-1　提示詞範本的準確使用方式與人機分工

範本不是問句，是契約書

提示詞範本可直接貼入 Mind AI 平台使用；範本中的【】欄位依專案替換，其餘結構不建議更動——範本的結構本身就是準確性的一部分。

S1 規格形萃取　SpecType Extraction

定位與本形運用

空間本形的「異同 → 分合 → 形」在此落地——把規格文本的差異，畫成形的邊界。

輸入

原始規格文件、需求訪談紀錄、既有形庫索引

動作

逐句辨識異同 → 分合切出候選形 → 屬性 / 屬性值建表 → 回填全體個體

輸出

SpecType 清單、屬性—屬性值表、萃取記錄

∩(T) 判準

每項需求皆被某個 SpecType 收容；無孤兒需求、無空白屬性。

操作程序　六拍

① 依「功能段落」切段，逐段標記所有可比較的差異點　② 差異點分組：同性質合、不同性質分，對照失敗才開新組　③ 每組取形名、編形 ID，撰寫 F(T) 與 ∩(T)　④ 建屬性—屬性值表，新屬性出現即回填全體　⑤ 與形庫比對：同 ∩(T) 標記複用並記差異，否則新建　⑥ 差異 → 組 → 形的推導鏈寫入證據欄



圖 5-2　S1 規格形萃取：輸入、動作、輸出與 ∩(T) 判準

檢核表　S 驗收前逐項打勾

- □ 異同清單涵蓋規格全部功能段落
- □ 每個 SpecType 皆有 F(T) 與 ∩(T)
- □ 屬性—屬性值表無任何空白欄位
- □ 與形庫比對完成（複用 / 新建皆有理由）
- □ 萃取記錄可回溯到規格段落編號

S2 介面形定義　InterfaceType Definition

定位與本形運用

「表頭即形」——訊號表的欄位是屬性、每一列是具體介面實體；X 軸在此把新介面掛進家族。

輸入

SpecType 清單、系統介面規範、既有介面家族

動作

定義訊號 / 參數欄位 → 設定方向與寬度 → 撰寫介面判準 → 家族比對命名

輸出

InterfaceType 訊號表、介面 ∩(T)、家族歸屬

∩(T) 判準

每個埠皆對應至少一項 SpecType 屬性；每項對外屬性皆有埠承載。

操作程序　五拍

① 由 SpecType 屬性表萃取「對外可見」的屬性，逐項決定承載它的訊號　② 建立訊號表：signal_name / direction / width / domain / reset_value 五欄必填　③ 撰寫介面 ∩(T)：協定型態、位元序、復位態、時脈域歸屬　④ X 軸家族比對：吻合則歸入並沿用命名慣例，不吻合才立新家族　⑤ 寬度、深度等可替換值一律進 ( ) 參數容器，不寫死

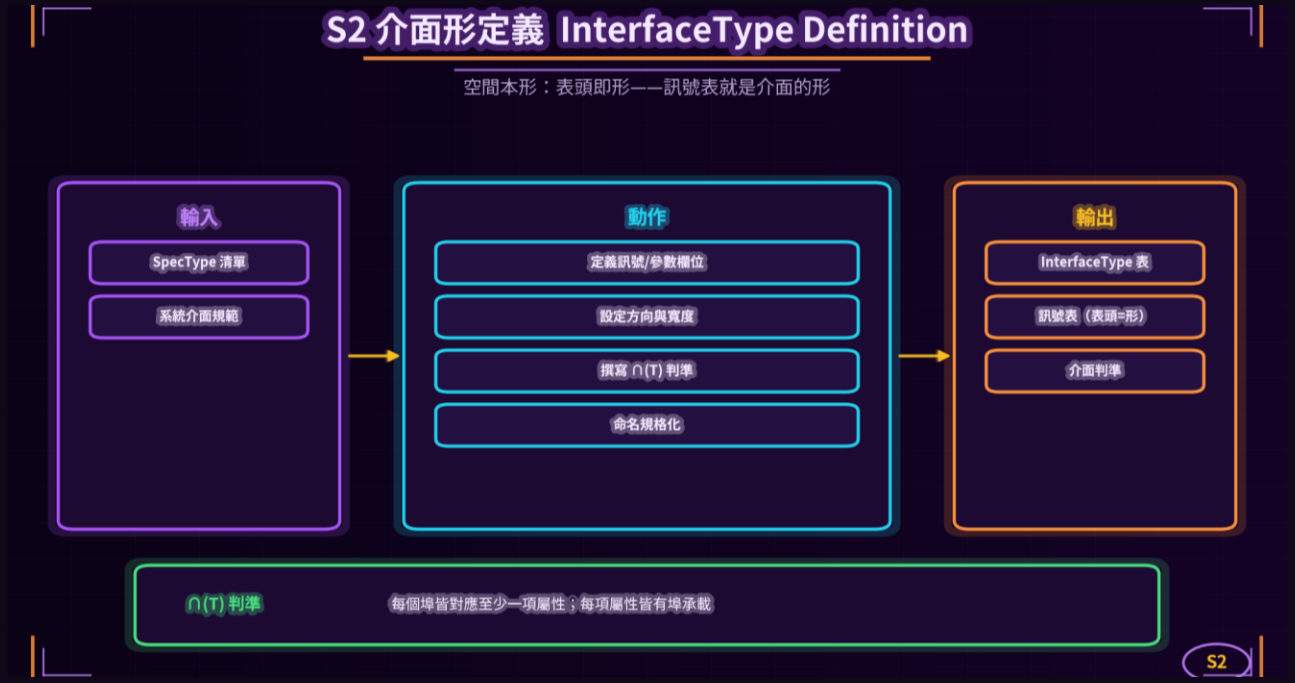


圖 5-3　S2 介面形定義：訊號表即介面的形

檢核表　S 驗收前逐項打勾

- □ 每個埠都標註來源 SpecType 屬性 ID
- □ 每項對外屬性都有埠承載
- □ 訊號表五欄 + 來源欄全填
- □ 家族歸屬有比對記錄與理由
- □ 可替換值已參數化為 ( ) 容器

S3 模組形分解　ModuleType Decomposition

定位與本形運用

Y 軸分解 + BOM / SOM——「拆下來之後，它還認得自己嗎」是唯一判準。

輸入

InterfaceType 訊號表、效能 / 面積 / 功耗約束

動作

Y 軸 BOM 分解 → SOM 依存標註 → 必要 / 選配判定 → X 軸家族比對

輸出

ModuleType 清單、BOM / SOM 結構表、依存關係圖

∩(T) 判準

拿掉任一必要件即動搖整體；選配件不影響模組身份。

操作程序　五拍

① 對整體模組做 Y 軸分解，逐層拆到「可獨立驗證的最小部件」　② 三類判定：可拆且認得→BOM；拆不下→屬性；可替換→( ) 參數容器　③ SOM 標註：部件間依存箭頭、拿掉後果、必要件 / 選配件　④ X 軸比對：相同 ∩(T) 直接實例化形庫既有 ModuleType　⑤ 產出 ModuleType 清單，主鍵 ModuleType_id、從屬 SpecType_id

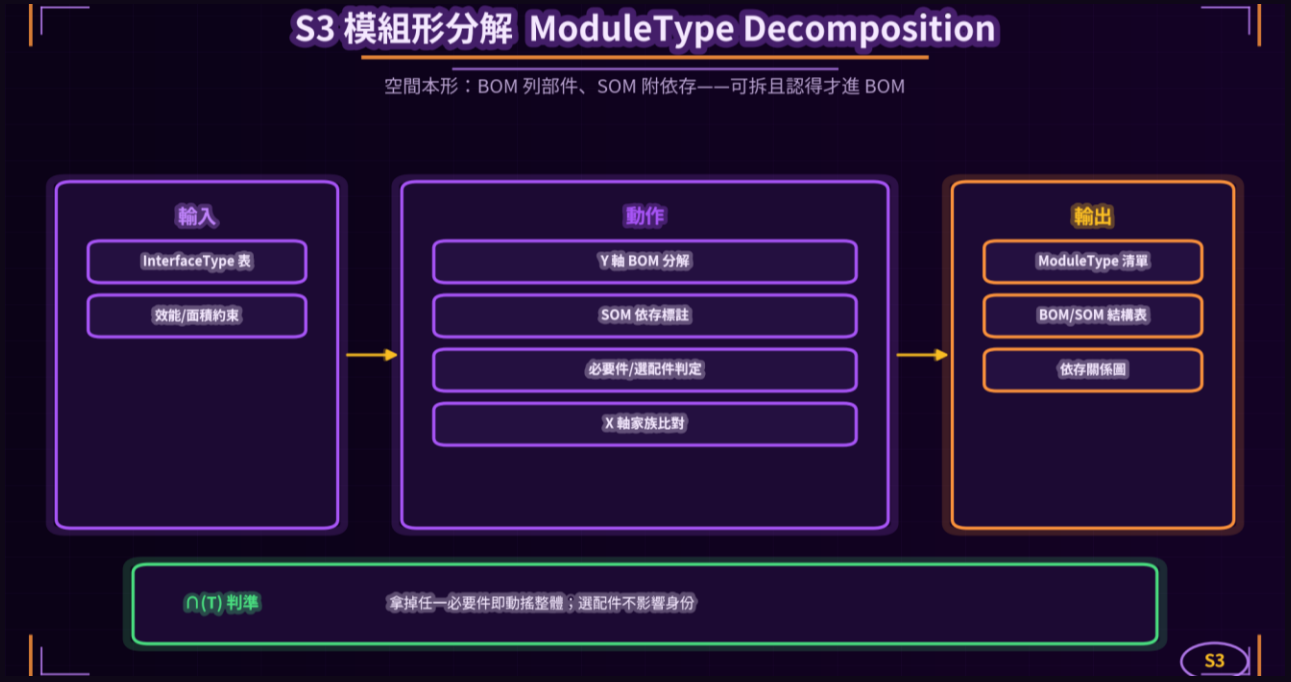


圖 5-4　S3 模組形分解：BOM 列部件、SOM 附依存

檢核表　S 驗收前逐項打勾

□ 分解終點皆達「可獨立驗證」粒度　□ 三類判定全員覆蓋，無元素未分類

□ SOM 依存表含拿掉後果　□ 必要 / 選配判定與 ∩(T) 一致

□ 子模組複用比對完成

S4 演繹骨架 Deductive Skeleton

定位與本形運用

Mind Model 的演繹推理在此成形——骨架的每條結論都必須能回溯到 ∩(T) 判準。

輸入

ModuleType 清單、各形 ∩(T) 判準表

動作

判準 → 骨架推導 → 模組層級展開 → 介面契約固化 → 時序骨架預排

輸出

演繹骨架文件、契約表、骨架一致性紀錄

∩(T) 判準

骨架每條結論皆可回溯至判準；無來源的結論必須刪除。

操作程序　五拍

① 逐條取 ∩(T) 特徵，推導「要滿足它，結構上必須存在什麼」　② 結論組織為模組層級圖：節點 = ModuleType、邊 = 契約　③ 固化介面契約四要素：訊號、時序、復位、例外行為　④ 預排時序骨架：狀態形清單、時脈域劃分、關鍵時序路徑　⑤ 一致性稽核：標不出來源的結論當場刪除，並列入「已刪除」清單



圖 5-5　S4 演繹骨架：自判準演繹，禁止自由聯想

檢核表　S 驗收前逐項打勾

- □ 每條結論都有判準來源編號
- □ 契約四要素齊備（訊號 / 時序 / 復位 / 例外）
- □ 模組層級圖與 BOM / SOM 一致
- □ 已刪除清單存在且被審視
- □ 時序骨架覆蓋全部時脈域

S5 時序形與 RTL 生成　TimingType & RTL

定位與本形運用

時間本形的「設計模式 + 邏輯生成」：先選 PatternType 模式，再把骨架翻譯成 Verilog；LLM 負責語言表面，結構由骨架決定。

輸入

演繹骨架、契約表、PatternType 模式庫、編碼規範

動作

選定編碼模式 → FSM / 時序形描寫 → 生成 Verilog RTL → 自我比對骨架

輸出

Verilog 程式碼、波形契約、生成記錄

∩(T) 判準

RTL 結構與骨架一一對應；時序契約與波形一致。

操作程序　五拍

① 為每個 ModuleType 選定 PatternType 模式卡（FSM、移位、除頻、FIFO）並記引用　② 依時序骨架描寫狀態形：狀態名、轉移條件、每狀態輸出契約　③ 生成 Verilog-2001：埠與訊號表逐欄對應，註解標形 ID 與模式 ID　④ 自我比對：RTL 層級 vs 骨架節點、RTL 契約 vs 契約表逐項對齊　⑤ 產出波形契約：關鍵訊號的期望波形文字化，交 S6 作驗證依據



圖 5-6　S5 時序形與 RTL 生成：模式化拼裝

檢核表　S 驗收前逐項打勾

- □ 每個模組都有模式選定記錄
- □ RTL 埠與訊號表逐欄一致
- □ 註解標齊形 ID 與模式 ID
- □ 骨架比對表全數勾稽
- □ 波形契約已文字化

S6 驗證形建構　VerifyType Construction

定位與本形運用

驗證形以「理想模型比對 + 誤注入」取證——TypeCov 的覆蓋目標直接對應 ∩(T) 的可觀察條件。

輸入
Verilog RTL、∩(T) 判準表、波形契約

動作
撰寫 testbench → 建立理想模型比對 → 誤注入反例 → 收集 TypeCov

輸出
驗證報告、TypeCov 覆蓋表、反例清單

∩(T) 判準
∩(T) 每項可觀察條件皆有覆蓋；反例皆被解釋。

操作程序　五拍

① 把 ∩(T) 的可觀察條件逐條改寫為覆蓋目標——TypeCov 的分子與分母　② 建立理想模型 scoreboard：以行為級模型作為「形的影子」逐筆比對　③ 誤注入：每條錯誤路徑至少一個反例（滿載、空載、break、NACK…）　④ 收集 TypeCov：已驗證條件 ÷ 全部可觀察條件，未覆蓋項列明細　⑤ 反例標根因（規格錯 / 骨架錯 / RTL 錯），修完重跑回歸

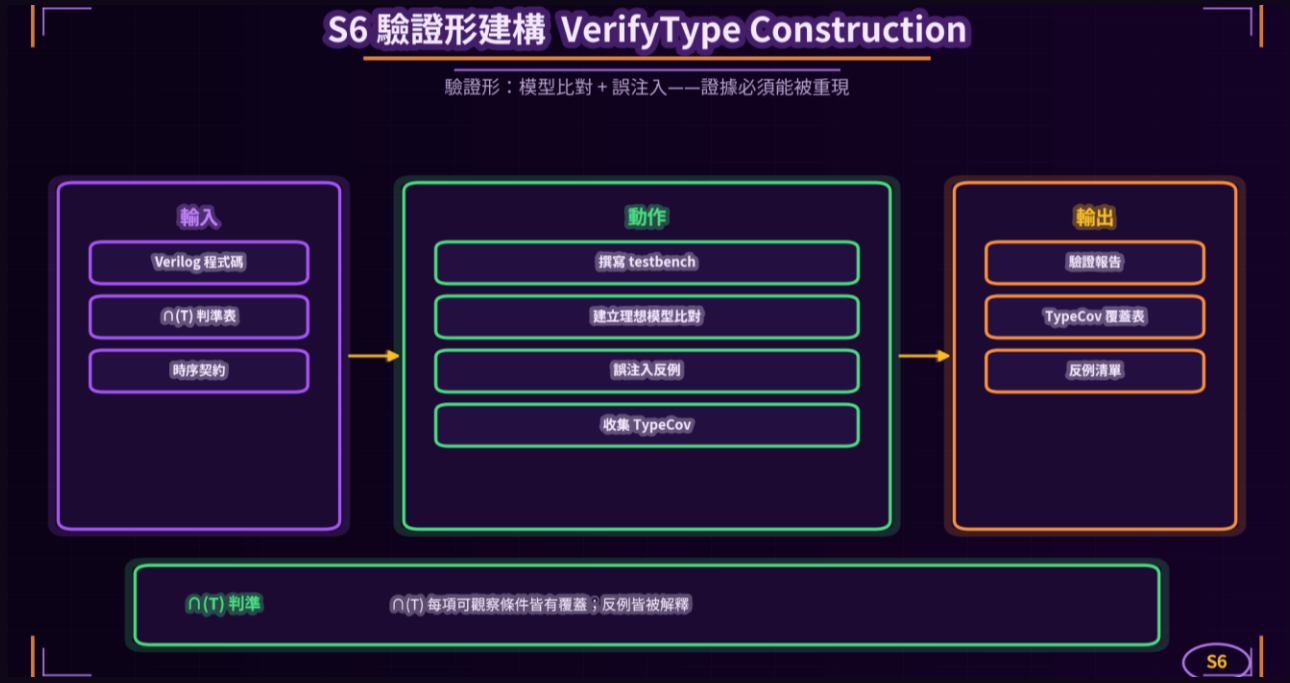


圖 5-7　S6 驗證形建構：模型比對、誤注入、覆蓋收集

檢核表　S 驗收前逐項打勾

□ TypeCov 分母 = ∩(T) 全部可觀察條件　□ 理想模型比對存在且逐筆運作
□ 每條錯誤路徑都有誤注入反例　□ FAIL 項皆有根因分類
□ 回歸重跑全綠——修到全綠才進 S7

S7 形證據鏈與自我演化 Evidence & Evolution

定位與本形運用

時間本形的「重構重組 + 持續學習 + 自我演化」：證據鏈補齊、形庫回填，本輪專案成為下一輪的起點。

輸入

驗證報告、全步驟證據、形庫現況

動作

補齊證據鏈 → 重構不改外在形 → 誤用沉澱檢核項 → 形庫回填版本化

輸出

完整形證據鏈、形庫增量、檢核表更新

∩(T) 判準

證據鏈無斷點；入庫的形皆附 ∩(T) 與證據。

操作程序　五拍

① 串鏈：S1 記錄 → S2 訊號表 → S3 BOM / SOM → S4 骨架 → S5 RTL → S6 報告，逐環點名　② 斷點處置：缺哪環補哪環，補不回者標記「經驗缺口」　③ 重構檢查：介面形與 ∩(T) 未變動，變動者回 S2 重走　④ 沉澱三件套：模式入 PatternType、誤用入檢核表、判準修訂入版本紀錄　⑤ 形庫版本化：新版本號、變更摘要、審核簽名，開放下一輪 S1 搜尋

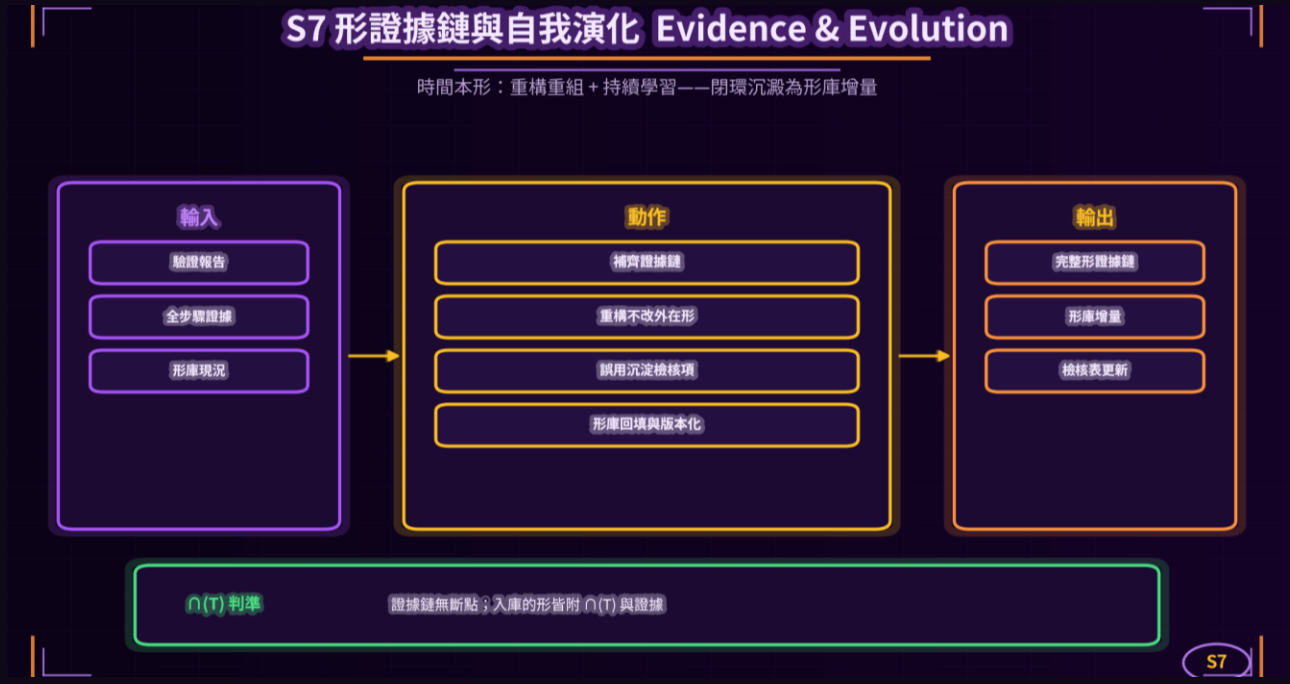


圖 5-8　S7 形證據鏈與自我演化：閉環沉澱

檢核表　S 驗收前逐項打勾

- □ 六環證據逐環點名完成
- □ 斷點全部處置或列缺口
- □ 重構稽核確認外在形未變
- □ 沉澱三件套齊備
- □ 入庫提案含版本與 ∩(T)

六模組 × 六形：每一份 RTL 都標明它示範哪個形

sync_fifo.v S3 / S5
ModuleType：BOM / SOM 樣板、( ) 參數容器

uart_tx.v S5
TimingType：FSM 四狀態形切分位元週期

uart_rx.v S5
TimingType：位元中點取樣 + 三點多數決

spi_master.v S5
InterfaceType：Mode 0 相位契約（CPOL=0、CPHA=0）

i2c_master.v S5
InterfaceType：開漏契約 + 四相位

timer.v S5
SpecType：可程式化時間容器

tb_sync_fifo.sv S6
VerifyType：理想模型比對 + TypeCov 四類全綠

tb_uart.sv S6
VerifyType：環回 + 誤注入反例（break、錯誤恢復）

契約的兌現 六個可綜合的 Verilog 模組與兩份 SystemVerilog 驗證平台，全文收錄。每支程式都標註它示範的形。

閱讀方式 把程式當「形的實體」而非單純程式碼：註解中的形 ID、模式 ID 對應第五章的步驟產出。



圖 6-1 Verilog 六模組 × 六形 對應總圖

六形各有歸宿，沒有一個形被遺忘在程式碼裡

sync_fifo 標 ModuleType、spi / i2c 標 InterfaceType、uart_tx / uart_rx 標 TimingType、timer 標 SpecType、兩份 tb 標 VerifyType——六形在一組範例裡同時在場，這就是「全程以形說話」的證據形態。

ModuleType 的部件依存 × TimingType 的狀態形切分



圖 6-2　sync_fifo 的形結構：四部件 SOM 依存

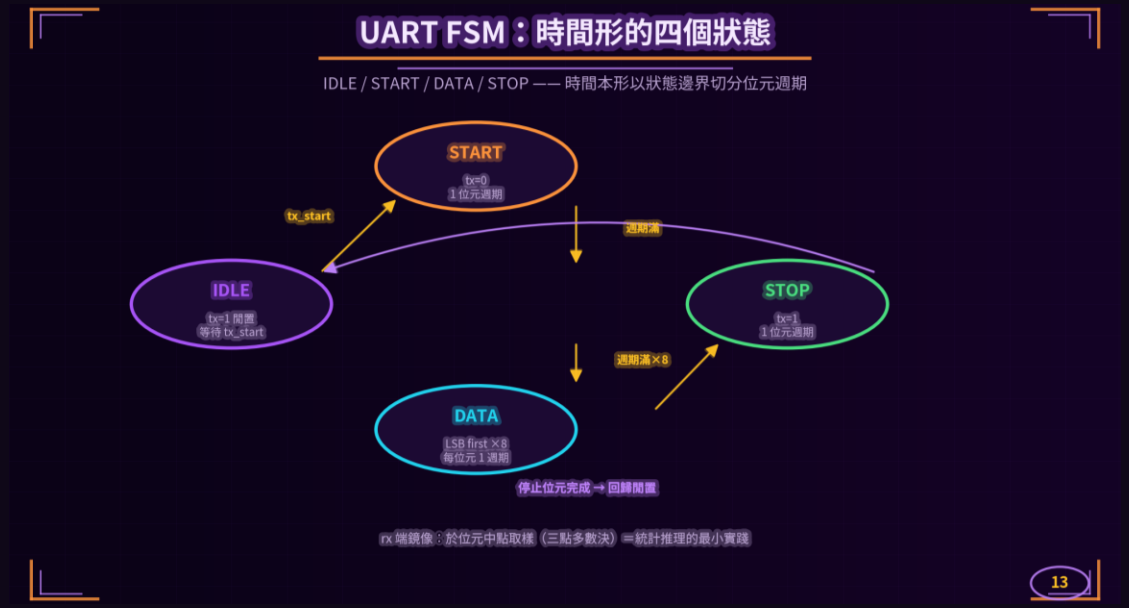


圖 6-3　UART FSM：IDLE / START / DATA / STOP 四個時間形

sync_fifo · v　ModuleType 的教學樣板

BOM　mem / wr_ptr / rd_ptr / cnt 四個可拆部件。

SOM　cnt 支撐 full / empty 判定；拿掉它，模組的「身份行為」塌陷——這就是必要件的定義。

( ) 容器　DATA_WIDTH、ADDR_WIDTH：換值不換身份。

∩ (T)　可同步讀寫 + 先進先出 + 旗標正確，方判入 SyncFIFOType。

uart_tx · v　時間本形的最小實踐

狀態形　時間被四個狀態形切分：IDLE / START / DATA / STOP。

時間容器　每一位元週期是同一個容器的重複刻度（CLKS_PER_BIT）。

資料鎖存　data_latch 體現「身份不因外部變動而漂移」。

PatternType　FSM + 位元計數 + 資料鎖存 三件套，形庫的標準卡。

取樣推理、邊界契約與一元宇宙四相位

6.4 uart_rx · v 把統計推理寫進硬體

雙正反器同步器
跨時域訊號：先畫邊界，再辨形。

三點過取樣多數決
多數決濾波的最小形態——假起始在此被拒絕。

位元中點取樣
把時間軸對齊到訊號的形上；rx_valid 是證據輸出點。

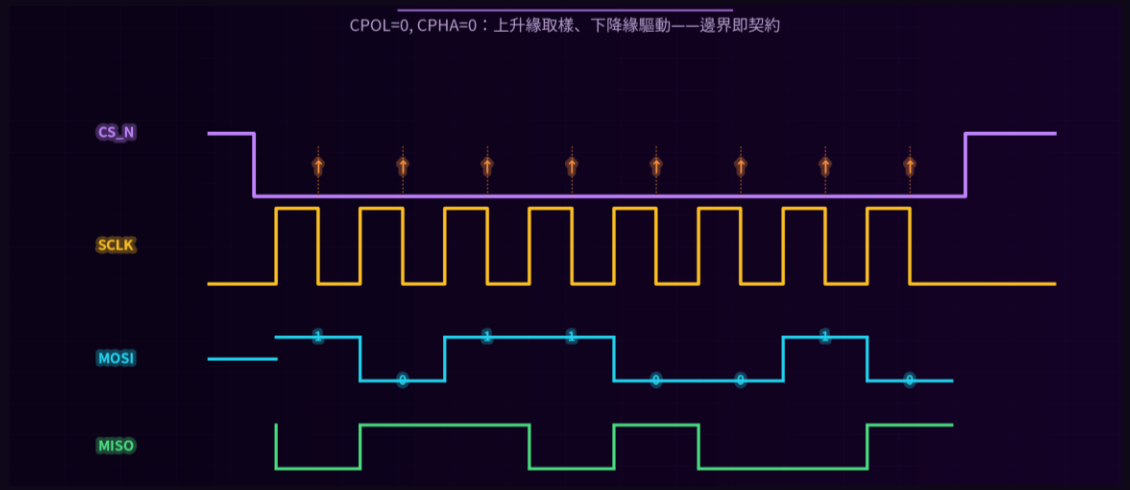


圖 6-4 SPI Mode 0 時序形：上升緣取樣、下降緣驅動



圖 6-5 I²C 時序形：START / WRITE / ACK / STOP

6.5 spi_master · v 介面形就是一份邊界契約

CS_N 的升降是傳輸框的邊界，SCLK 的邊緣是資料的刻度。∩(T) = CPOL=0 + CPHA=0 + MSB first；實作以除頻器 + 雙向移位暫存器的 PatternType 拼裝。

6.6 i2c_master · v 最細的時間形切分

一位元切成 setup / rise / high / fall 四相；SDA 只允許在 SCL 低期間變動，START / STOP 是唯一例外邊界。開漏模型體現容器語意：誰都可以拉低，沒有人可以強拉高。

時間容器與驗證平台：裝什麼不換身份，比什麼才有證據



圖 6-6 timer：load_val 是容器刻度，身份不隨載入值改變

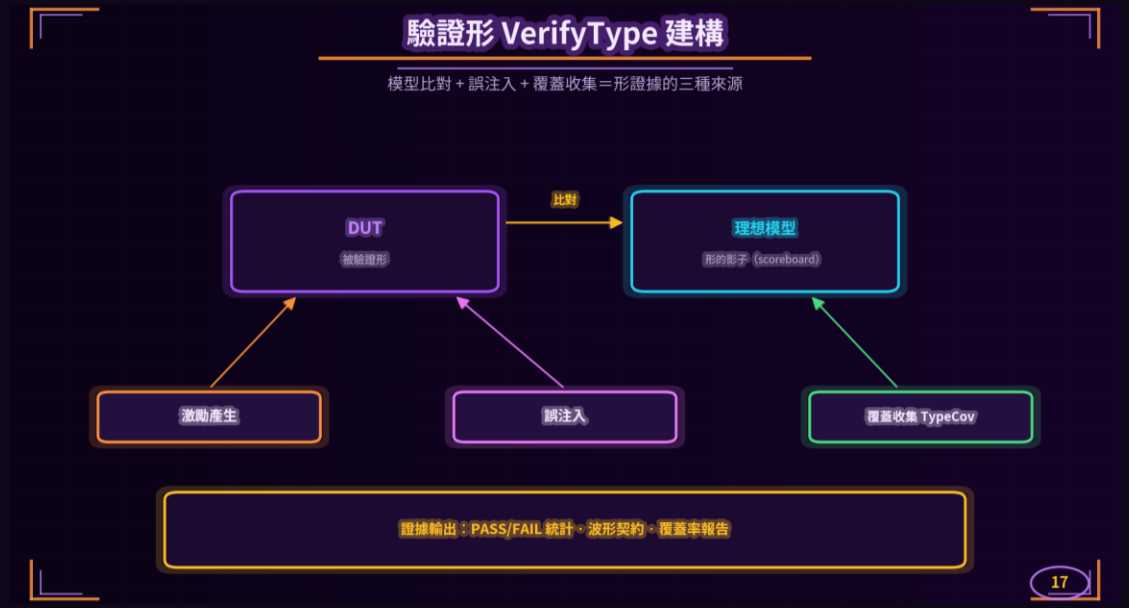


圖 6-7 驗證形建構：理想模型比對 + 誤注入 + 覆蓋收集

6.7 timer.v | ( ) 容器的活教材

load_val 是裝進時間容器的內容物，換值不換身份。

periodic 於載入鎖存：模式即身份的一部分，於形成立時定案；單次模式不因輸入毛刺重複觸發。

irq 為可觀測證據點；週期模式自動重載，自帶重複證據流。

6.8 驗證形 VerifyType 的兩個樣板

tb_sync_fifo 以佇列理想模型逐筆比對；四類覆蓋目標就是 ∩(T)：FIFO 序、滿載拒寫、空載拒讀、同時讀寫計數不變。

tb_uart 以環回閉環取證、以 break 誤注入驗證錯誤路徑；err_seen 黏著旗幟 = 證據必須能被重現。

建議順序：先跑 tb_sync_fifo 建立信心，再跑 tb_uart 涵蓋錯誤路徑。

形覆蓋率 TypeCov：對判準收斂，不對測試數虛胖

定義刻意簡單

TypeCov = 已驗證可觀察條件數 ÷ 全部可觀察條件數

分母來自 ∩(T)：覆蓋率永遠對著判準收斂，而不是對著「跑過的測試數」成長。

使用規則三條

① 未覆蓋項必須列明細，並附風險評估。

② 100% 只代表「判準全被驗過」，不代表判準本身正確——那是第一層的責任。

③ 反例數量與覆蓋率同為報表必填欄位。

在四層保證中的位置

TypeCov 是第三層（形式驗證・S6）的核心產出。

通過標準 = 回歸全綠且反例全解釋；不通過則反例入清單、循環修。

S7 串鏈階段，TypeCov 報告與反例清單隨證據索引一同入庫。

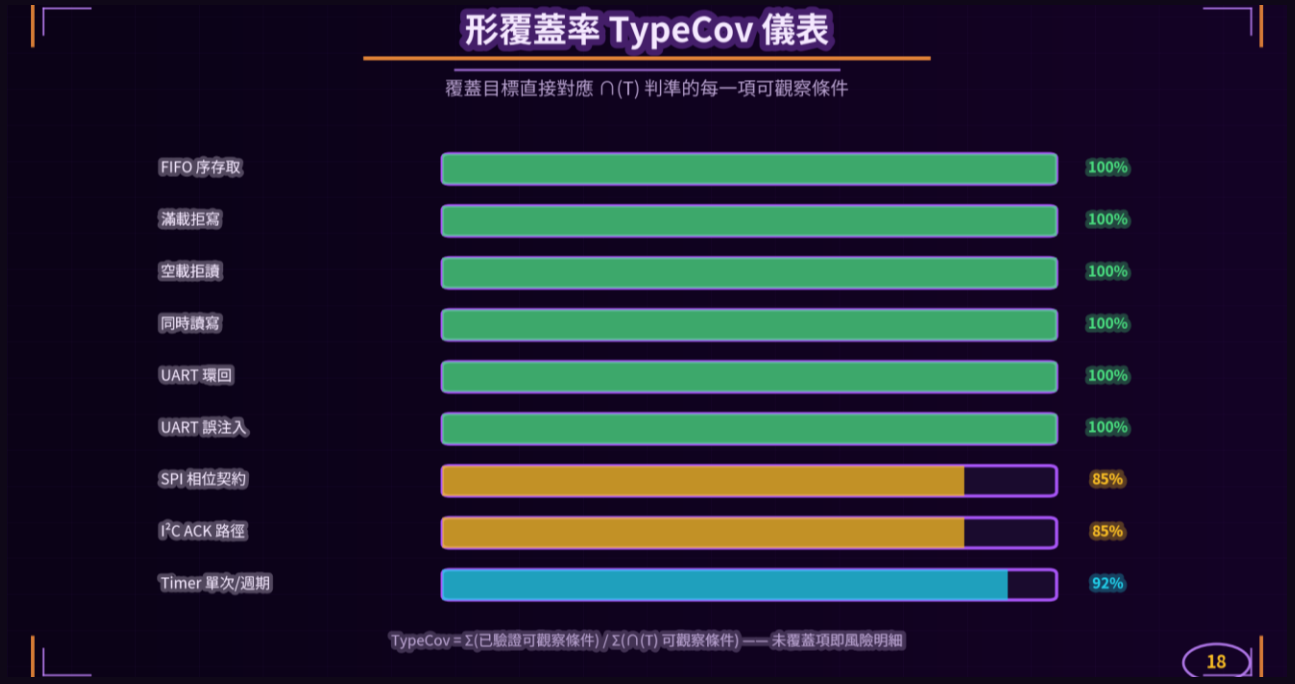


圖 7-1 形覆蓋率 TypeCov 儀表：覆蓋目標對應 ∩(T) 可觀察條件

分母就是 ∩(T) 的可觀察條件

tb_sync_fifo 四類：FIFO 序、滿載拒寫、空載拒讀、同時讀寫計數不變；tb_uart 兩類：資料位元組環回 × 錯誤路徑（break）。儀表全綠，是每個 ∩(T) 條件都有證據可言。

形證據鏈：七環相扣、可回溯、可審計

S1 萃取 異同清單、差異→組→形推導鏈 ▶ 萃取記錄

S2 介面 訊號—屬性映射、家族比對 ▶ 訊號表 + 比對表

S3 分解 三類判定、必要件後果 ▶ BOM/SOM 表

S4 骨架 判準→結論推導、已刪除清單 ▶ 演繹紀錄表

S5 生成 模式引用、骨架比對表、波形契約 ▶ 生成記錄

S6 驗證 TypeCov 報告、反例清單含根因 ▶ 驗證報告

S7 串鏈 六環點名、形庫版本 ▶ 證據索引

七環皆為必答：每一環都留下可指名、可核對的載體。

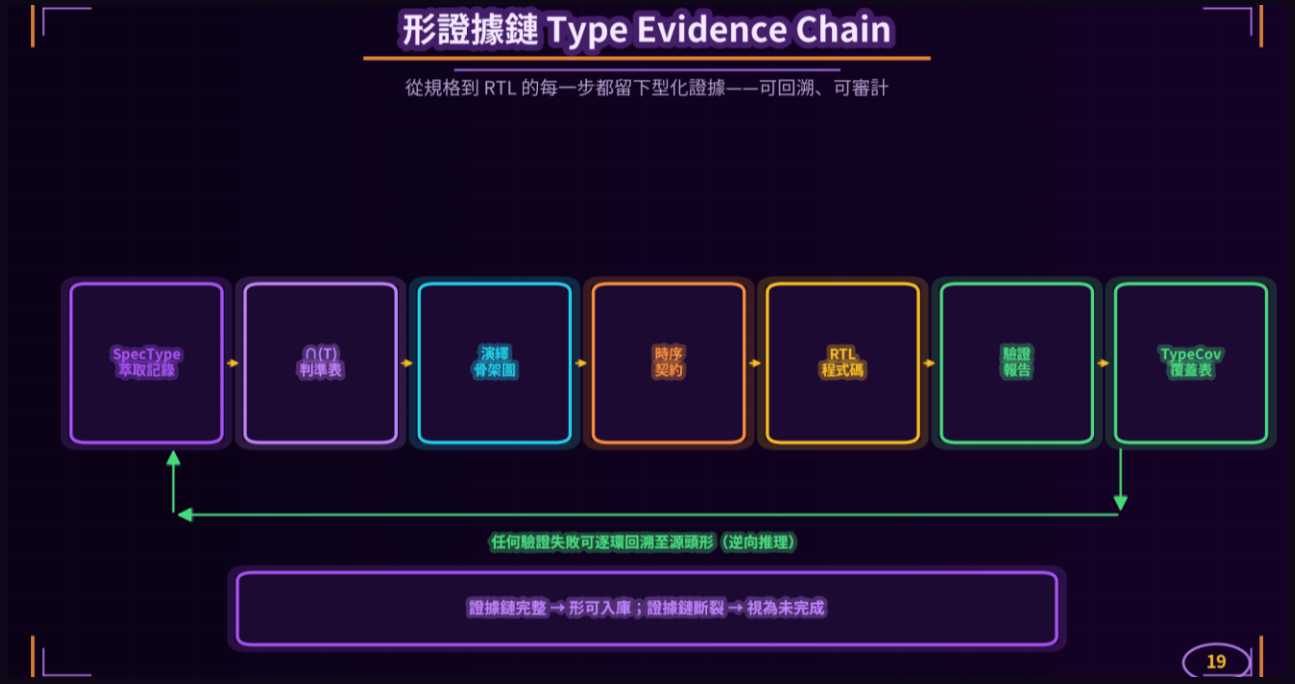


圖 7-2 形證據鏈：可回溯、可審計

證據鏈的稽核要件

第四層保證在 S7 落地：通過標準是「六環無斷點」，不通過則「缺哪環補哪環」。

斷點讓整份形提案不可簽核：證據索引與版本紀錄是串鏈的最後組件，也是入形庫的門票。

可回溯 = 能沿鏈還原每一步產出；可審計 = 每一環都有載體可指名。

誤用對照總表：每次評審直接掃描的負面清單

① 叫 LLM 直接寫 RTL、不給判準
後果：生成漂移、無驗收標準　糾正：S4 骨架先行、判準先行

② 把列舉清單當形
後果：封閉、無法判入新成員　糾正：改用 ∩(T) 生成規則

③ 用 Form 代指「形」
後果：術語體系污染　糾正：一律 Type，當場更正

④ 參數寫死、把參數當部件
後果：( ) 容器拍平、形膨脹　糾正：參數化 + 三類判定重來

⑤ 只驗 happy path
後果：錯誤路徑盲區　糾正：誤注入強制進檢核表

⑥ testbench 拄 RTL 寫法
後果：同構盲點　糾正：獨立理想模型比對

⑦ 重構時順手改介面
後果：驗證作廢、回歸斷裂　糾正：外在形凍結，動介面回 S2

⑧ 無證據入庫
後果：形庫污染、下輪踩坑　糾正：缺證據標缺口，不入庫

⑨ 混步下提示詞
後果：輸出無法驗收　糾正：一步一範本

⑩ 以名稱認家族 / 複用
後果：錯複用、形同實異　糾正：以 ∩(T) 比對認形

讓方法落地：角色分工、範本與形庫治理

形主 Type Owner

主責 S1、S7

裁決 ∩(T) 爭議、審核入庫。

交付：形庫品質、判準版本

設計工程師 Designer

主責 S2–S5

向形主申請形變更。

交付：訊號表、骨架、RTL

驗證工程師 Verifier

主責 S6

回報反例根因分類。

交付：testbench、TypeCov、反例清單

平台管理員 Platform Admin

跨步職責

維護提示詞範本與形庫環境。

交付：範本版本、存取稽核

協作節奏：形驅動衝刺

一輪 S1–S7 即一個衝刺；每日站會以形報進度——「ST-003 已過 S4」比「做了 80%」有效。衝刺檢視必備三件套：TypeCov 報告、反例清單、證據鏈稽核表。

8.2 提示詞範本治理

版本化　範本與程式碼同級管理；修改必留紀錄與理由。

凍結區 / 活區　【角色】【任務】【輸出格式】【禁止事項】凍結，改動需形主核准；【輸入】為活區，每專案替換。

誤用反饋　每次被糾正的用法，回寫為禁止事項——範本隨誤用進化。

8.3 形庫治理：入庫與出庫

入庫門檻　∩(T) 齊備 + 證據鏈無斷點 + 形主簽核，三者缺一不可。

出庫機制　連續兩輪未被複用且判準被修訂推翻的形，標記「休眠」；休眠不刪除，保留演化軌跡。

8.3 形庫治理：版本與變更

版本節奏　每輪衝刺結案出一個形庫版本；版本摘要就是團隊的知識增量聲明。

變更不對稱　新增列隨時可做、新增欄必須評審——守護「表頭即形」的穩定。

成熟度演進：四級路線，不追求一次到位

L1

手動形驅動

特徵

人肉執行七步驟、形庫為試算表

升級條件

七步驟走通一個完整專案

L2

範本化

特徵

S1–S7 範本全員使用、檢核表強制

升級條件

連續兩輪零混步、零 Form 誤用

L3

證據化

特徵

TypeCov 成常規、證據鏈可稽核

升級條件

形庫版本化運行滿三輪

L4

自演化

特徵

模式自動萃取建議、誤用自動沉澱檢核項

升級條件

形庫增量中機器貢獻占比過半

導入建議：走通一遍，勝過半遍三次

不要追求一次到位。第一個專案選一個中等複雜度的介面型 IP（如本書 UART 子系統規模），完整走通 S1–S7 一遍，比三個專案各走半遍有價值得多。

那「走通的一遍」會直接產出——

- 屬於你們自己的形庫雛形
- 檢核表初版
- 第一批模式卡

不給判準，就沒有準確

形不是清單，而是生成規則 ∩(T)；準確不是口號，而是可舉證的四層證據鏈。

方法 · S1–S7 七步驟

萃取→介面→分解→骨架→生成→驗證→串鏈：一步一範本、一步一檢核，混步即退件。

資產 · 形庫與模式卡

∩(T) 齊備、證據鏈無斷點、形主簽核——三件套是入庫門票；形庫版本就是團隊的知識增量。

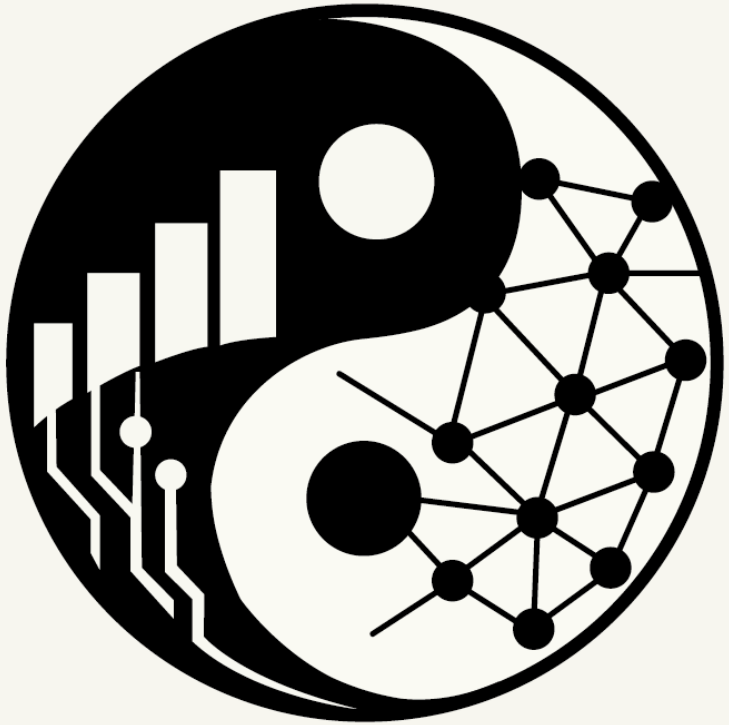
路線 · L1 → L4

從手動形驅動出發，走過範本化、證據化，邁向自演化；第一個專案走通一遍，勝過半遍三次。

交付與附錄

全書八支 Verilog / SystemVerilog 檔案隨附《IC 設計大腦_Verilog 程式碼.zip》，可直接投入模擬；附錄 A 為 S1–S7 提示詞範本總表，附錄 B 為檔案清單。

LonGreat 願景：創造理解世界本質的智慧生命



MIND AI

「讓人工智慧不只是工具，而是真正具備理解、推理、學習與創造能力的智慧生命體系。」

從知識統一，到思維統一，最終邁向智慧統一。這就是 LonGreat 所追求的 AGI 永恆之道。

www.longreat.net

張長豐
gamma@livemail.tw

張椁嶺
mindaiworks@gmail.com